SMART HOME AUTOMATION

Final Project Report | Software Design Patterns | Java 17

Project type: team of two | Version: 2.0 | Date: November 2026

1. Introduction

Smart Home Automation is a simulated console application that controls a Zigbee-style light, a thermostat, and a smart door lock. The application also publishes motion and temperature sensor events. The objective is to demonstrate the coordinated use of six design patterns in one maintainable Java project, rather than separate disconnected examples.

2. Requirements and Scope

The instructor requires a coordinated project with at least six distinct design patterns, at least two individually implemented patterns per student, Java or Go code, screenshots, UML diagrams, introduction, body, conclusion, and further work. This implementation targets those deliverables. Physical Zigbee hardware and network transmission are outside the scope of this simulation.

3. Pattern-to-Feature Mapping

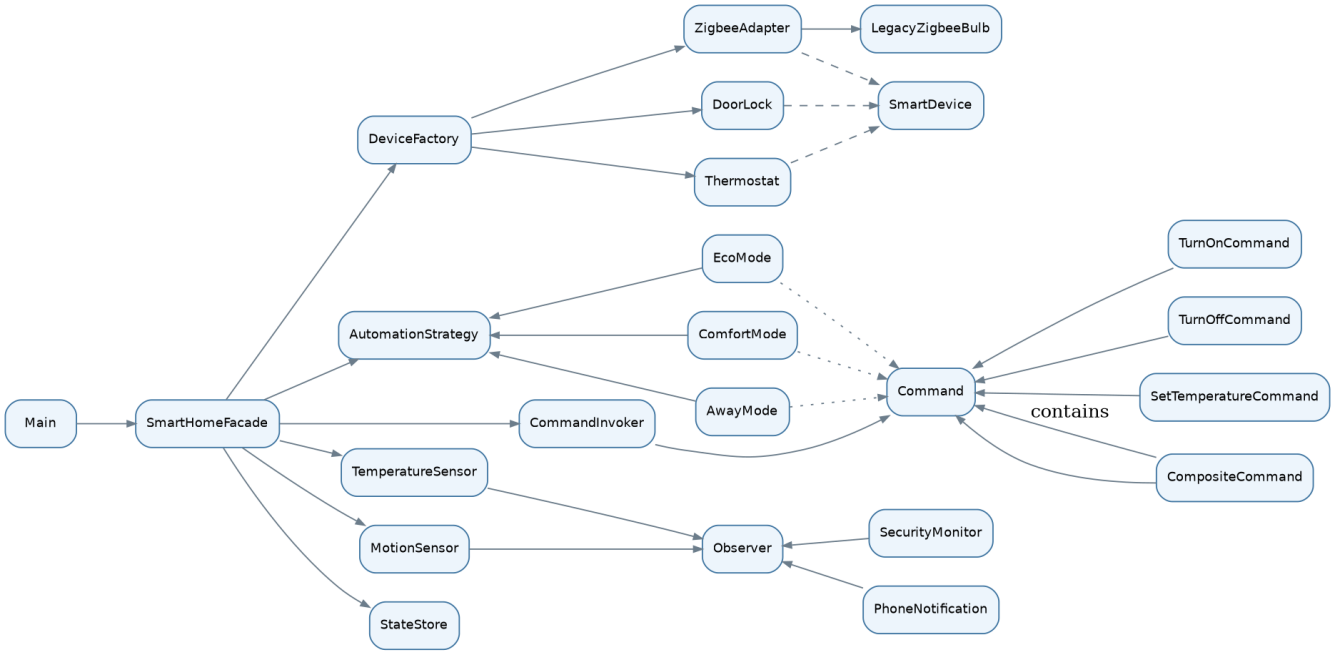
Pattern	Implementation	Demonstration
Observer	MotionSensor, TemperatureSensor, observers	Sensor events notify subscribers
Command	Invoker, ON/OFF, temperature, composite	Undo restores previous states
Strategy	EcoMode, ComfortMode, AwayMode	Runtime selected home modes
Adapter	ZigbeeAdapter, LegacyZigbeeBulb	Legacy bulb follows SmartDevice
Simple Factory	DeviceFactory	Creates devices by type
Facade	SmartHomeFacade	CLI calls one coordinator

Terminology: DeviceFactory is a Simple Factory, not a subclass-based GoF Factory Method. It still demonstrates centralized object creation; discuss this distinction accurately during defense.

4. System Architecture

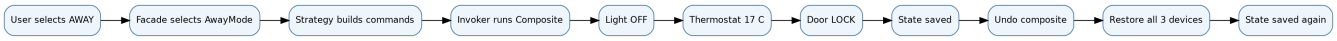
The Main class handles text input and uses SmartHomeFacade as the only gateway to business operations. The facade creates devices with DeviceFactory, uses CommandInvoker for reversible state changes, and selects an AutomationStrategy at runtime. An Adapter wraps the simulated legacy bulb. Sensors implement Subject, and notification classes implement Observer. StateStore persists device state between launches.

5. UML Class/Relationship Diagram



An editable PlantUML source file is included as docs/uml/class-diagram.puml. The rendered diagram on this page summarizes the relationships.

6. UML Interaction Diagram



The detailed sequence is also supplied as docs/uml/sequence-diagram.puml. An automation mode executes as one composite command; Undo processes child commands in reverse order.

7. Implementation Details

Observer: MotionSensor notifies SecurityMonitor and PhoneNotification. TemperatureSensor notifies PhoneNotification and labels readings over 30 C as high-temperature alerts. List-copy iteration allows notification without exposing the internal observer list.

Command: Each command records its previous state. CompositeCommand groups a mode into one undoable action and reverses its children in reverse order. Invalid temperature input is rejected before adding history.

Strategy: ECO sets the thermostat to 19 C and secures the door; COMFORT sets it to 22 C and switches the light on; AWAY sets it to 17 C, switches off the light, and locks the door. Every strategy returns commands rather than modifying devices directly.

Adapter: ZigbeeAdapter translates SmartDevice.turnOn()/turnOff() into the older powerOn()/powerOff() API. This is a simulated protocol boundary.

Factory and Facade: DeviceFactory centralizes creation. SmartHomeFacade orchestrates creation, strategies, commands, sensors, Undo, and persistence; Main never accesses those components directly.

Persistence: StateStore uses a Java Properties file at data/smarthome.properties by default. Device states survive a restart; command history intentionally belongs to the current session only.

8. Demonstration and Application Screenshots

Run com.smarthome.Main, choose 8 for status, 1 for light ON, 5 to unlock the door, 3 then 25 to set temperature, 7 then 3 for AWAY, 8 to inspect, 9 for Undo, and 8 to inspect restored values. Choose 6 for motion or 10 then 35 to simulate a temperature alert. Restart and choose 8 to verify saved state.

```
Actual console run (Java 17)
Choose an option: Zigbee bulb: power ON.
Choose an option: Front door is UNLOCKED.
Choose an option: Enter temperature (16-30): Thermostat temperature set to 25 C.
Mode: Zigbee bulb: power OFF.
Thermostat temperature set to 17 C.
Front door is LOCKED.
AWAY mode activated.
Front door lock: LOCKED
Choose an option: Front door is UNLOCKED.
Thermostat temperature set to 25 C.
Zigbee bulb: power ON.
Last command was undone.
Front door lock: UNLOCKED
```

Actual console output was captured from an executable Java run. The accompanying source files allow the demonstration to be repeated live.

9. Automated Testing

The project includes 15 JUnit 5 test cases covering default state, individual Undo, repeated commands, door lock state restoration, temperature validation, composite Undo, invalid modes, all three strategies, state reload, sensor events, factory creation, legacy adaptation, observer removal, and reverse-order Undo. Run using mvn clean test or IntelliJ Maven test execution.

10. Suggested Team Contributions

Student A: Observer, Command, Strategy. Student B: Adapter, Simple Factory, Facade. This is a suggested division for a team of two, not a statement of verified authorship. Both students should be prepared to demonstrate at least two patterns and explain how their classes integrate.

11. Conclusion

The smart-home simulation combines six design-pattern roles in a cohesive Java application. Its main improvement over disconnected examples is the shared Facade and Command flow: actions initiated by selectable strategies are undoable, devices are constructed through a factory, a legacy interface is adapted, and sensor events reach subscribed observers. File persistence preserves the visible device state across restarts.

12. Further Work

Potential extensions include physical Zigbee or MQTT connectivity, an HTTP or graphical user interface, more sensors, scheduled automation rules, user authentication, audit history, persistent Undo history, and improved validation of corrupted persistence data. These are future directions, not features currently delivered.

References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

2. Freeman, E., & Robson, E. (2020). Head First Design Patterns (2nd ed.). O'Reilly Media.

3. Course document: Final Project Defense Requirements (provided by instructor).